Resenha

On the Criteria to Be Used in Decomposing Systems into Modules

David L. Parnas

Projeto de Software

Rafael Caetano da Silva

Belo Horizonte, MG – Março de 2026

O artigo On the Criteria to Be Used in Decomposing Systems into Modules, de David L. Parnas, é um dos textos mais importantes da engenharia de software, principalmente quando se fala em modularização. Nele, o autor discute como sistemas devem ser divididos em módulos e critica a forma tradicional como essa divisão costuma ser feita. A principal ideia do artigo é que a forma como escolhemos dividir um sistema influencia diretamente na sua qualidade, manutenção e evolução.

Logo no início, Parnas explica que a modularização é importante porque ajuda a melhorar três aspectos principais: a organização do desenvolvimento, a flexibilidade do sistema e a facilidade de entendimento. A ideia é que, ao dividir um sistema em partes menores, diferentes equipes podem trabalhar de forma mais independente, o sistema fica mais fácil de modificar e também mais simples de compreender.

No entanto, o autor chama atenção para um problema: normalmente se fala muito sobre dividir sistemas em módulos, mas quase nunca se discute qual critério deve ser usado para fazer essa divisão. E é exatamente isso que o artigo busca responder.

Para explicar melhor, Parnas apresenta um exemplo de sistema chamado KWIC, que basicamente recebe linhas de texto, gera variações dessas linhas e depois organiza essas variações em ordem alfabética. A partir desse exemplo, ele mostra duas formas diferentes de dividir o sistema em módulos.

A primeira forma organiza os módulos de acordo com as etapas do processamento. Ou seja, existe um módulo para entrada de dados, outro para gerar as rotações, outro para ordenar e outro para saída. Esse tipo de divisão é bastante comum porque segue a lógica de um fluxograma, onde cada etapa do processamento vira um módulo.

Já a segunda forma, considerada mais inovadora pelo autor, organiza os módulos com base no conceito de information hiding. Nesse caso, cada módulo é responsável por esconder uma decisão importante de projeto. Por exemplo, o modo como os dados são armazenados fica escondido dentro de um módulo específico, e os outros módulos não precisam saber como isso funciona internamente.

A grande diferença entre essas duas abordagens aparece quando o sistema precisa ser modificado. Parnas mostra que, na primeira decomposição, qualquer mudança em decisões importantes pode afetar vários módulos ao mesmo tempo. Já na segunda abordagem, essas mudanças ficam isoladas dentro de um único módulo, o que torna o sistema muito mais fácil de manter .

Outro ponto interessante discutido no artigo é a questão da independência entre módulos. Na decomposição tradicional, os módulos dependem muito uns dos outros, pois compartilham detalhes internos como estruturas de dados. Já na abordagem baseada em ocultação de informação, os módulos se comunicam apenas por meio de interfaces bem definidas, o que reduz o acoplamento e facilita o desenvolvimento em paralelo.

Além disso, Parnas também comenta sobre a compreensibilidade do sistema. Na primeira abordagem, para entender um módulo, muitas vezes é necessário entender outros módulos também, já que tudo está interligado. Na segunda abordagem, cada módulo pode ser entendido de forma mais isolada, o que facilita bastante o aprendizado e a manutenção do sistema.

Um dos principais conceitos apresentados no artigo é justamente o de que um bom critério de modularização não deve ser baseado na sequência de execução do programa, mas sim nas decisões de projeto que podem mudar ao longo do tempo. O autor defende que devemos identificar essas decisões e escondê-las dentro de módulos específicos.

Parnas também discute alguns detalhes mais técnicos, como o impacto da modularização na eficiência do sistema. Ele reconhece que a abordagem baseada em ocultação de informação pode, em alguns casos, gerar perda de desempenho, principalmente devido à maior quantidade de chamadas entre módulos. No entanto, ele sugere que isso pode ser resolvido com técnicas de implementação mais eficientes, sem abrir mão dos benefícios da modularização.

Outro ponto interessante é a ideia de estrutura hierárquica entre módulos. O autor explica que sistemas bem estruturados costumam ter módulos organizados em níveis, onde módulos de nível mais alto dependem dos de nível mais baixo. Isso permite que partes do sistema sejam reutilizadas ou até removidas sem afetar o restante.

Na conclusão, Parnas reforça sua principal crítica: não devemos começar o projeto de um sistema pensando em fluxogramas, como era comum na época. Em vez disso, devemos pensar nas decisões mais importantes e nas partes do sistema que podem mudar no futuro, e usar isso como base para definir os módulos .

De forma geral, o artigo é bastante importante porque introduz o conceito de information hiding, que até hoje é fundamental na engenharia de software. Muitas práticas modernas, como encapsulamento em programação orientada a objetos e design de APIs, têm relação direta com essa ideia.